

ECE 315

Steven Knudsen, PhD, PEng

Lecture 2 :

- Embedded Systems overview



UNIVERSITY
OF ALBERTA

Last Lecture

- Course introduction and overview

This Lecture

- Embedded Systems overview
 - Importance of embedded systems
 - Examples
- Challenges
 - SWaP
 - Constrained “everything”

Notice

Lectures may be audio recorded for the purpose of a student's individual study as part of an approved academic accommodation.

Learning Objectives

- Know the main aspects of this course, what technologies and knowledge are needed to work with computer interfaces, especially in an embedded environment.
- Appreciate the pervasiveness of embedded computing
- Be able to recall several application areas
- Define what “real-time” means
- Differentiate between hard- and soft-real time systems and needs
- Recall some examples of past and current embedded systems

High-level View

Main focus of the course covers embedded systems, including

- Microcomputers and hardware interfaces
- Software design for real-time embedded systems
- Computer communication interfaces
- Interfacing to sensors and actuators

Some assumptions...

- Know some C/C++
- Know Git and are happy to use Github
- Some idea of what an operating system (OS) is and does
 - Linux, Windows, Mac (OSX)
 - OS provides common support for applications
- Some idea of computer communications
 - Wired and wireless networking
 - Internet
 - Inter-computer communications

What is an Embedded System?

An embedded system is a special-purpose computer system designed to perform a dedicated function or a specific set of tasks, often within a larger mechanical or electrical system.

- *ChatGPT*

An embedded system is a computer system that is used to monitor and control a product or engineering system. An embedded system often has only a simplified and/or an application-specific user interface. A conventional PC or tablet user interface is usually absent in an embedded system.

- *Dr. Bruce Cockburn, ECE 315 W2025*

What is an Embedded System?

- Increasingly the lines are blurred
 - “Dumb” systems are getting embedded processors
 - Multiple small embedded systems comprise very large systems that are, arguably, embedded – they have no user interface, autonomous to some degree
- But there are also incredibly small systems - Texas Instruments MSPM0C1104, a Cortex-M0+

<https://www.engadget.com/science/space/spacex-will-attempt-starships-11th-flight-test-on-monday-204237829.html>



Why Embed a Processor?

- Increasingly inexpensive, so why not?
 - RP2040, a Cortex-M0+ processor that runs at 133 MHz and is 32-bit is \$CA1
 - Other processors can be as low as a few pennies in quantity
- Distributes processing
 - Reduced single-point failure risk
 - Greater flexibility – e.g., can change one small part of a system without having to re-program the main part
- High-speed signal processing for better sensing, can compensate for sensor inaccuracies, shifts due to environment, aging, etc.
- Can improve function via upgrades (not the pennies processors!)

Embedded Processor Applications

- Consumer Electronics
 - Phones, entertainment systems, game, home networking, cameras,...
- Household Electronics
 - Microwave ovens, refrigerators, thermostats, high-efficiency furnaces, washer/dryer, security systems, solar power, ...
- Vehicles
 - Motor functions like ignition control, fuel injectoion timing, electrical power generation and distribution, fault detection and diagnosis, "black box" event recording, anti- lock braking, entertainment systems, anti-theft, wireless locking & ignition, maps & navigation, driving assistance, ...
- Industrial
 - infrastructure (e.g., communications, electrical power, banking system), instrumentation, factories, robots, numerical controlled tools, refineries, oil fields, gas plants,



Adapted from ECE 315 W2025 Dr. Bruce Cockburn

11

Real-Time Embedded Systems

- Primarily focused on **real-time embedded systems** in this course
- Real-time system is fast enough to react to changes in its environment to control something **correctly**, **efficiently**, and **safely**.
- A **hard real-time embedded system**;
 - Violations of real-time constraints are unacceptable
 - Constraint violations could cause injury or death, equipment damage, loss of material, and/or serious financial loss
- A **soft real-time embedded system**;
 - Occasional violations of the real-time undesirable, but can be tolerated
 - E.g., a vending machine or an Automated Teller Machine (ATM) can tolerate small delays with only minimal occasional inconvenience to human users.



Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

12

These definitions are subjective in the sense that the time scale must be defined.

RFID example – 30 μ s response time

Performance Measures

- **Algorithm correctness**: Do all possible inputs produce the desired system state changes and outputs?
- **Response time**: Does the system respond fast enough to input/external changes?
- **Predictable or deterministic response time**: Is the response time sufficiently predictable (that is, have a small variance) as well as being acceptably fast?
- **Resource efficiency**: Was the consumption of power, natural gas, water, other chemicals, communication bandwidth, etc. minimized when accomplishing the work?

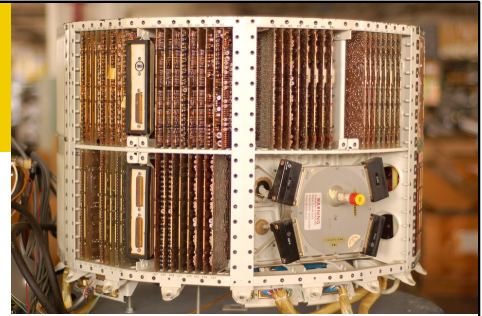
Other Measures

- **SWaP**
 - Size, weigh, and power appropriate?
- **Cost**
 - Initial development costs and later system costs appropriate?
 - Total lifetime cost acceptable?
- **Well-designed and built**
 - Will the system operate properly over its entire lifetime?
 - Will it be easy to maintain, update, repair? Does it need to be?
- **Standards and best practices compliant**
 - Is it safe, complies with regional laws, etc.

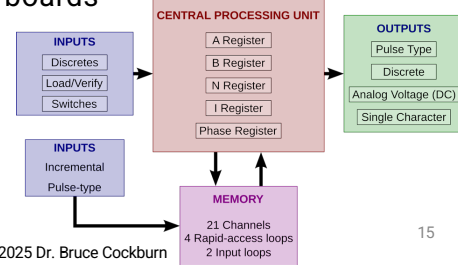
Early Embedded Systems



- D-17B (D17B) computer was used in the Minuteman I NS-10Q missile guidance system (1962)
 - 62 pounds (28 kg),
 - 1,521 transistors, 6,282 diodes, 1,116 capacitors, and 5094 resistors
 - Mounted on 75 double copper-clad, engraved, gold-plated, glass fiber laminate circuit boards
- The high degree of reliability and ruggedness of the computer were driven by the strict requirements of the weapons system.



D-17B Functional Location of Arithmetic Registers and Rapid-Access Memory Loops



Wikipedia contributors. "D-17B." *Wikipedia, The Free Encyclopedia*. Wikipedia, The Free Encyclopedia, 31 Oct. 2024. Web. 13 Oct. 2025.

Adapted from ECE 315 W2025 Dr. Bruce Cockburn

15

Embedded Systems

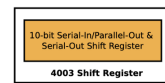
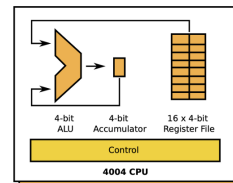
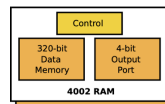
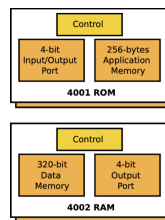
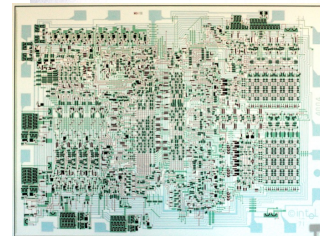
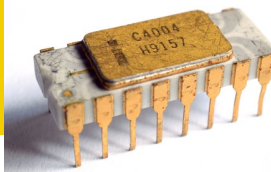
Apollo Guidance Computer (AGC)

- Most computers in the 1950s and early 60s were room-sized
- AGC needed to be small, as small as possible
- ~32 kg and 16,400 cm³
- 15-bit computer(!)
- It has just 36K words of ROM (read-only memory) and 2K words of RAM (erasable memory)
- Intel 4004 was still 10 years away...



Embedded Systems – Intel 4004

- One of the most famous first microprocessors is the Intel 4004, November 1971
- 4-bit processor
- Then, a high-level of integration
 - D-17B; 1,521 transistors, 28 kg, \$US2 million
 - AGC; ~16,800 transistors, 32 kg, \$US1.5 million
 - 4004; ~2,300 transistors, 1-2 g, \$US466



A digression...

UofA main computer – late 70s through 1984

- Computer that everyone used on campus was the mainframe
- Housed in General Services Building



https://www.chessprogramming.org/Amdahl_470

UofA main computer – late 70s through 1984

- Computer that everyone used on campus was the mainframe
- Housed in General Services Building
- Connect via dedicated frontend computers to dumb terminals

<https://en.wikipedia.org/wiki/index.php?title=PDP-8&oldid=1315947782>



 UNIVERSITY OF ALBERTA

<https://en.wikipedia.org/wiki/index.php?title=PDP-11&oldid=1316101157>



<http://dunfield.classiccmp.org/tty/index.htm>



https://www.chessprogramming.org/Amdahl_470

Amdahl 470/V6 vs Arduino

- Amdahl 470/V6
 - Performance: 3.5 MIPS
 - Architecture: 32-bit CPU
 - Registers:
 - 16 general-purpose registers
 - 8 floating-point registers
 - 8 scratchpad and control registers
 - Main Memory: 1 to 8 million bytes (1MB to 8MB)
 - High-Speed Buffer: 16,384 bytes (16KB)
 - Power Consumption : Crazy!
- ~\$3 million in 1979 – about \$14 million today
- Arduino R3
 - Performance: 20 MIPS
 - Architecture: 8-bit CPU
 - Registers:
 - 32 general-purpose registers
 - Main Memory:
 - 32 kB program/data
 - 2 kB SRAM
 - Power Consumption : 275 mW
- \$CA25-35 depending on vendor

Amdahl power requirements

- The Amdahl 470 processors are air cooled and require a minimum of 12.5 tons of air conditioning and 4890 cubic feet of air per minute. Minimum BTU output for a 470 processor is 79,000 BTUs per hour. Figures are generally higher depending on the processor model and amount of memory installed. Power must be available to the Amdahl 470 power distribution unit from two sources, 415 and 60 Hz. Both sources must be four wire and three phase at 208 volts.
- 79,000 BTU is about 23 kWatt (!)

...digression over



UNIVERSITY
OF ALBERTA

23

Embedded Applications – Supervisory Control and Data Acquisition Systems (SCADA)

- SCADA stands for Supervisory Control and Data Acquisition.
- It is a computer-based system used to monitor and control industrial processes across large-scale or distributed environments such as factories, power plants, water treatment facilities, and transportation networks.
- A SCADA system is an industrial control system (ICS) that gathers real-time data from remote sensors and equipment, processes that data, and enables supervisory control – allowing operators to monitor, analyze, and manage operations from a central location.

SCADA – Core Components

Sensors and Actuators:

- Measure process variables (temperature, pressure, flow, voltage) and perform control actions (open/close valves, start/stop motors). Increasingly have embedded processors

Remote Terminal Units (RTUs) or Programmable Logic Controllers (PLCs):

- Embedded processor unit that interfaces with sensors and actuators in the field; collect data and execute control commands.

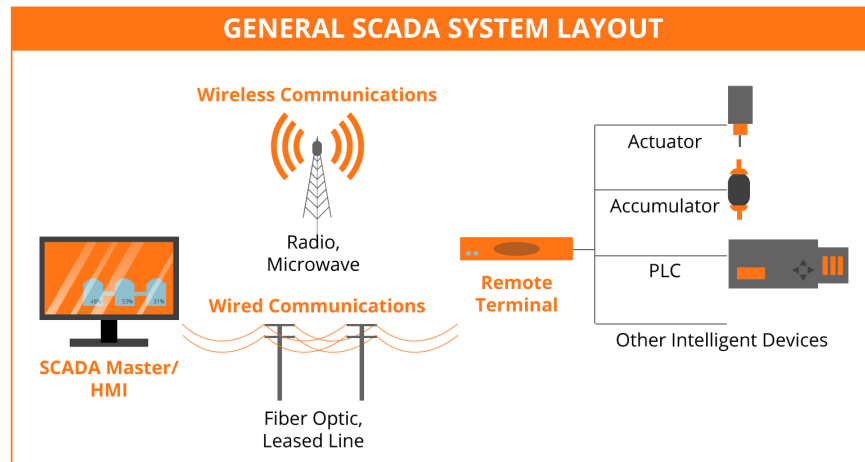
Communication Network:

- Connects RTUs/PLCs with the central control system, often using industrial communication protocols (Modbus, DNP3, OPC, Ethernet/IP).

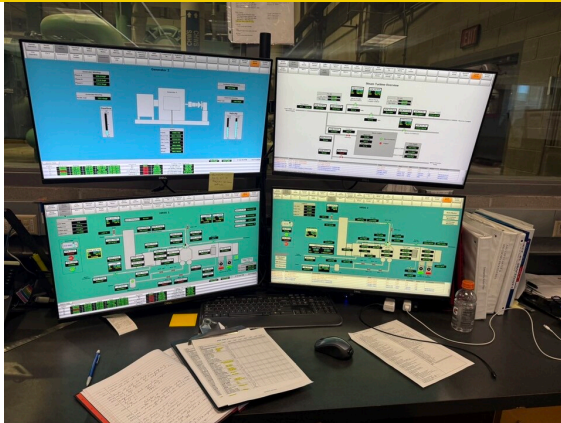
SCADA Master Station / HMI (Human–Machine Interface):

- Central servers and graphical user interfaces that visualize data, log events, and allow operators to issue control commands.

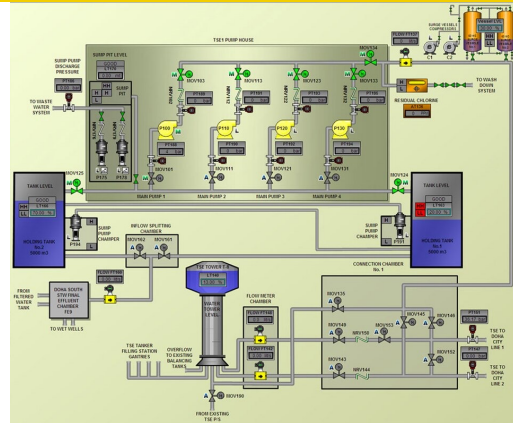
SCADA - Example



SCADA - Example



Master Station



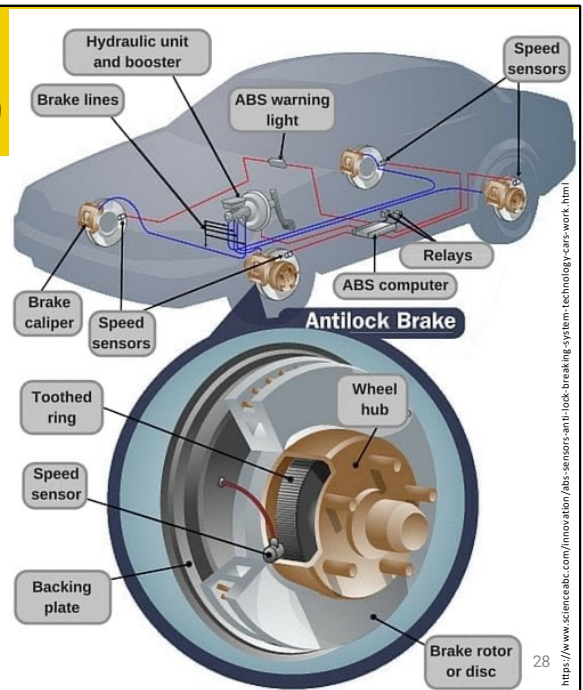
Pumping Station



"File:Chiller plant SCADA system.jpg." *Wikimedia Commons*. 15 Feb 2025, 20:50 UTC.
https://commons.wikimedia.org/w/index.php?title=File:Chiller_plant_SCADA_system.jpg&oldid=998836148> 13 Oct 2025, 19:21.
 "File:SCADA PUMPING STATION 1.jpg." *Wikimedia Commons*. 22 Jul 2024, 01:51 UTC.
https://commons.wikimedia.org/w/index.php?title=File:SCADA_PUMPING_STATION_1.jpg&oldid=90188712> 13 Oct 2025, 19:18.

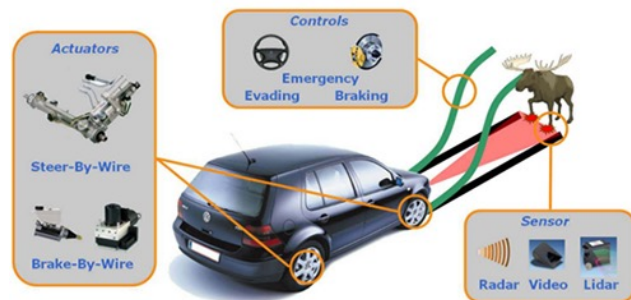
Embedded System Example – Anti-lock Braking System (ABS)

- Anti-lock Braking Systems have been around for quite a while now
- It is a feedback control system that is tuned to a specific auto make
- Recently, has been augmented by other sensors and systems



Smart(er) ABS

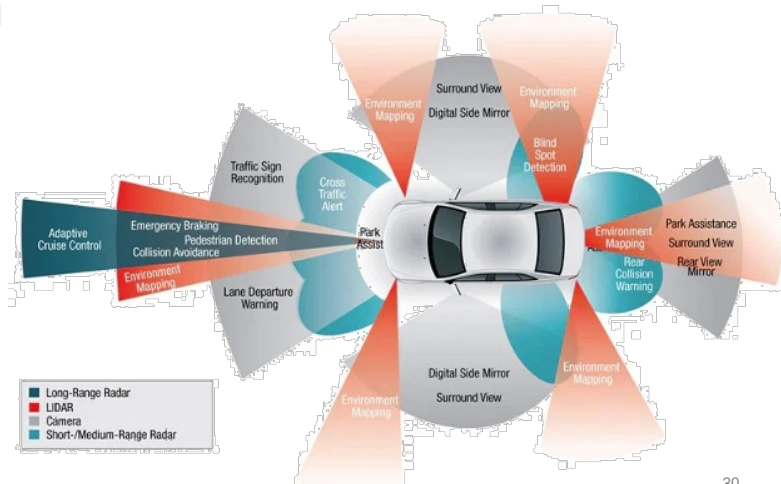
- Automatic Emergency Braking System
- Combines more sensors to help control the action of the brakes
- Note that there is also avoidance using steer-by-wire
- Tricky because you can't steer if the brakes are locked up



<https://tinyurl.com/543nr2pz>

Smarter Cars

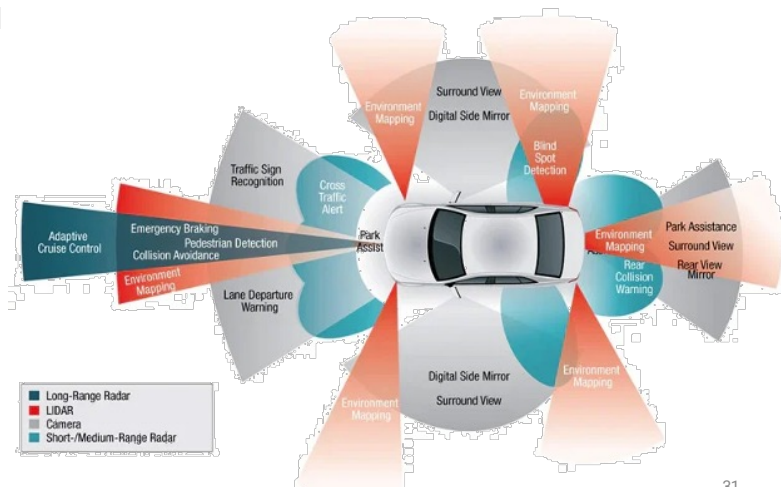
- Multiple entertainment and convenience systems.
- Systems monitoring (engine, suspension, etc.)
- Anti-lock braking systems, electronic stability control mandatory
- Automatic emergency braking becoming standard
- Next will be connected, then automated vehicles



Smarter Cars

VEHICLES TODAY HAVE FROM 100-200 EMBEDDED PROCESSORS, WHICH WILL ONLY INCREASE

- Multiple entertainment and convenience systems.
- Systems monitoring (engine, suspension, etc.)
- Anti-lock braking systems, electronic stability control mandatory
- Automatic emergency braking becoming standard
- Next will be connected, then automated vehicles



Embedded Applications – Internet of Things

- The Internet of Things (IoT) is almost by definition networked embedded devices
- Even those things like sensors that may not be directly connected in the sense that we can directly access them, are made available by an embedded processor with network access
- An interesting development has been the specialized protocols that are designed for IoT, such as MQTT (Message Queuing Telemetry Transport), BLE (Bluetooth Low Energy)
 - We will visit these later

Embedded Challenges

- (Almost) Everything is constrained
 - Might have only kB of RAM, not GB
 - Might have only MB of FLASH, not TB
 - Processor clock of MHz, not GHz
 - No dedicated floating-point processing
 - Limited operating system, if any
- Do have
 - Lots of I/O
 - Smart I/O -- peripheral devices do things for you, e.g., DMA, SPI, I2C, ADC
 - Low cost

Embedded Challenges - SWaP

SWaP – Size, Weight, and Power

Embedded Challenges - SWaP

SWaP == Size, Weight, and Power

SWaP-C == SWaP + Cost

Embedded Challenges - SWaP

SWaP == Size, Weight, and Power

SWaP-C == SWaP + Cost

- The most important aspect of embedded application is the selection of the processor. Ideally, want
 - Lowest cost in volume
 - Computational and storage capacity that is 2x or 3x what is anticipated
 - Lowest possible power consumption (...unless power is abundant, as it might be on vehicles, for example)
 - Maybe special characteristics – automotive grade, large temperature range, ...

Next Lecture

- Microprocessor/controller architecture review
- Examples relevant to ECE 315

Questions

